



Tribhuvan University
Faculty of Humanities and Social Sciences

Project Proposal on
AI-Based Recruitment Platform

Submitted to:
Department of Computer Application
Birendra Multiple Campus
Bharatpur, Chitwan

In partial fulfillment of the requirements for the Bachelors in Computer Application

Submitted by:

Name: Krishna Shrestha

Semester: Eight(8th)

Registration Number:6-2-19-115-2021

Date: 2082-12-30

Name: Manjila Adhikari

Semester: Eight(8th)

Registration Number:6-2-19-119-2021

Date: 2082-12-30

1. Introduction

The AI-Powered Recruitment Management System is a web-based application designed to streamline and enhance the recruitment process through automation and intelligent decision-making. The system connects recruiters and candidates on a unified platform, enabling efficient job posting, application handling, and candidate evaluation. Unlike traditional recruitment platforms, this system integrates machine learning and Natural Language Processing (NLP) techniques to analyze resumes and rank candidates based on their relevance to job descriptions. By transforming unstructured resume data into structured insights, the platform enables data-driven hiring decisions. The system aims to reduce manual effort, improve candidate selection accuracy, and provide a scalable solution for modern recruitment challenges (Ramos, 2003).

2. Problem Statement

Recruitment processes in many organizations are inefficient, time-consuming, and heavily dependent on manual effort. The major issues identified are:

- ✧ Manual screening of resumes is time-consuming and inefficient, especially when handling large volumes of applications.
- ✧ Lack of intelligent systems to match candidates with job requirements, leading to difficulty in identifying suitable applicants quickly.
- ✧ High chances of human bias and errors in candidate selection, affecting fairness and decision quality.
- ✧ Existing recruitment platforms lack automated ranking mechanisms, making it difficult to prioritize candidates based on relevance.

3. Objectives

The objectives are designed to directly address the identified problems in a priority-driven manner:

- ✧ To automate the resume screening process by extracting and processing resume data, thereby reducing manual effort and improving efficiency.
- ✧ To develop an AI-based candidate-job matching system using NLP techniques such as TF-IDF and cosine similarity to accurately compare resumes with job descriptions.
- ✧ To minimize human bias and improve decision accuracy by implementing a data-driven candidate evaluation and ranking mechanism.

- ✧ To design and implement an automated candidate ranking system that prioritizes applicants based on relevance scores, enabling recruiters to identify the best candidates quickly.

4. Methodology

As the requirements are well-known, clear, have fixed time and project is small, Iterative waterfall model is the most suitable method for the project.

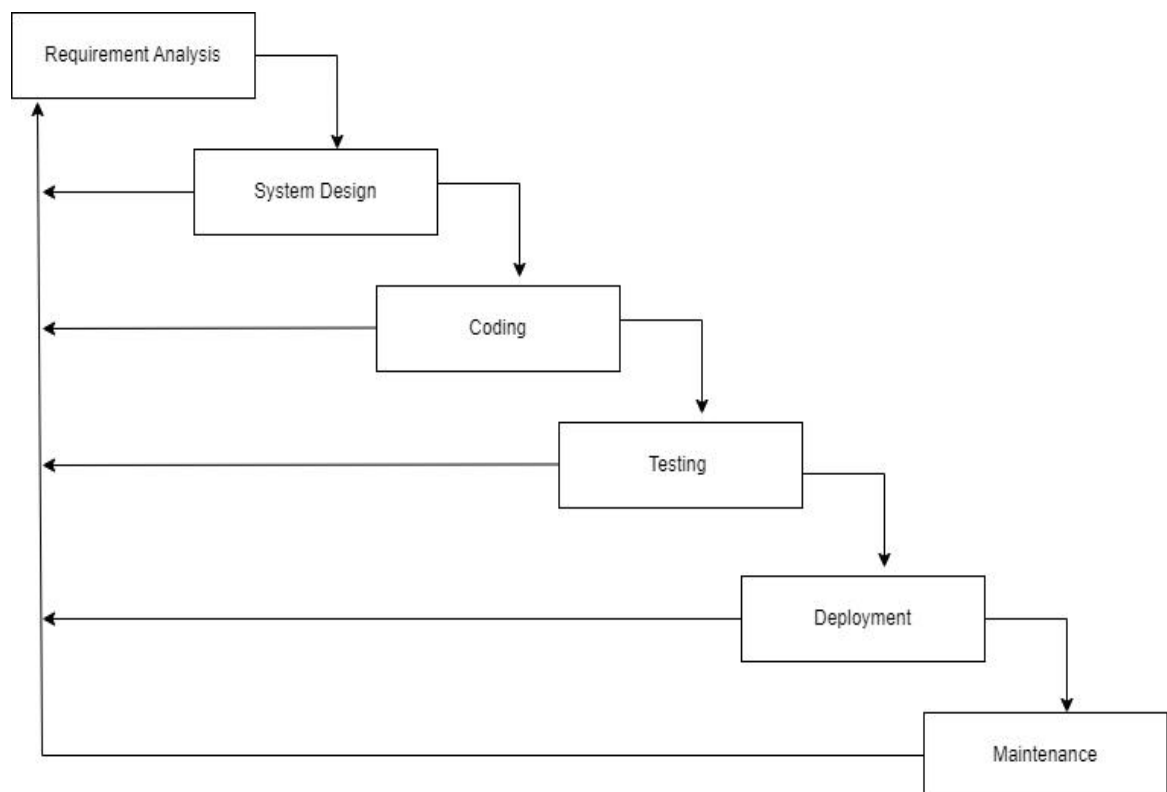


Fig 1: Iterative Waterfall Model

4.1 Requirement Identification

4.1.1 Study of Existing System

Existing recruitment platforms such as: LinkedIn, Kumari Jobs, CloudFactory provide job posting and application features but lack efficient automated candidate evaluation and ranking mechanisms. These systems rely heavily on manual screening, which reduces efficiency and scalability.

4.1.2 Literature Review

Studies on Natural Language Processing (NLP) highlight its effectiveness in extracting meaningful information from unstructured text such as resumes. Techniques such as TF-IDF and cosine similarity are widely used for document comparison and relevance scoring. Research indicates that automation in recruitment significantly reduces time, improves consistency, and minimizes bias (Manning et al., 2008).

4.1.3 Requirement Analysis

Functional Requirement:

- ✧ User Registration & Login with role-based access (Admin, Recruiter, Candidate)
- ✧ Job Posting (create, update, delete job listings)
- ✧ Resume Upload (PDF format)
- ✧ Resume Processing and text extraction
- ✧ Candidate Ranking based on similarity scores
- ✧ Job Application and status tracking
- ✧ Admin Dashboard for system monitoring and user management.

Non-Functional Requirement:

- ✧ Performance: Efficient handling of multiple users and large datasets
- ✧ Usability: Simple and intuitive user interface
- ✧ Security: Secure authentication and password encryption
- ✧ Scalability: Ability to support future feature enhancements

4.2 Feasibility Study

4.2.1 Technical

Back-end development using Flask or Django with object-oriented design

Integration of machine learning libraries such as scikit-learn, NLTK, or spaCy

Development of ML models in Jupyter Notebook and integration into the application

Implementation of resume parsing and text processing modules

4.2.2 Operational

- ✧ Provides a user-friendly interface for all roles
- ✧ Enables recruiters to manage job postings and applicants efficiently
- ✧ Allows candidates to apply and track applications easily

- ✧ Improves overall recruitment workflow through automation..

4.2.3 Economic

- ✧ Utilizes open-source tools, reducing development cost
- ✧ Minimizes manual recruitment effort, saving time and resources
- ✧ Cost-effective compared to traditional recruitment systems
- ✧ Low maintenance and scalable for future upgrade.

4.3 High Level Design of System

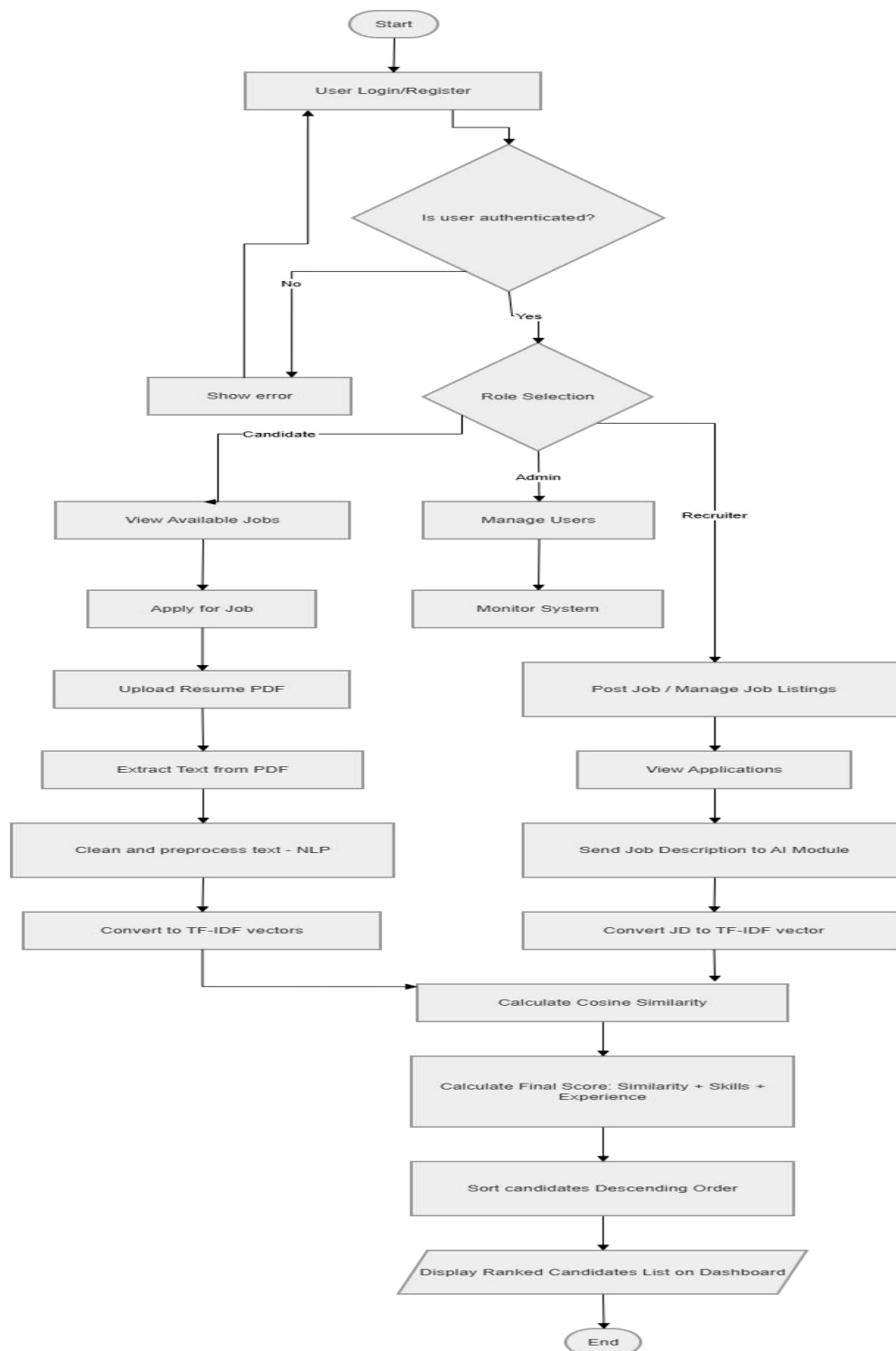


Fig 3: System Flowchart of Online Share Portfolio Management System

4.3.1 Description of Algorithms

1. Sorting Algorithm (TimSort – Python Built-in):

TimSort is a hybrid sorting algorithm based on Merge Sort and Insertion Sort, designed for efficient real-world performance. Implementation:

The system uses Python's built-in `sorted()` function to arrange candidates based on their final scores in descending order, ensuring that the most relevant candidates are displayed first.

2. Searching Mechanism (Database Querying & Filtering):

Efficient searching is achieved using database queries that filter records based on user-defined criteria. Implementation:

The system retrieves jobs, candidates, and applications using filtering conditions such as skills, job titles, and application status. While simple searches resemble linear search, database optimization techniques ensure scalability and performance.

3. Machine Learning Algorithms (Core AI Module):

a. TF-IDF (Feature Extraction):

Converts textual data (resumes and job descriptions) into numerical vectors based on term importance.

b. Cosine Similarity (Similarity Measurement):

Measures the similarity between resume vectors and job description vectors to determine relevance.

c. Combined Ranking Score (Enhanced System Design):

The final candidate score is calculated using multiple factors:

- ✧ Similarity Score (from cosine similarity)
- ✧ Skill Match Score
- ✧ Experience Weight

This produces a more accurate and interpretable ranking system. user's data

5. Gantt Chart

Week	1	2	3	4	5	6	7	8	9	10	11	12
	(01/01	(01/08	(01/15	(01/22	(02/03	(02/10	(02/17	(02/24	(03/01	(03/08	(03/15	(03/22
Activity	01/07)	01/14)	01/21)	01/28)	02/09)	02/16)	02/23)	02/30)	03/07)	03/14)	03/21)	03/28)
Requirement analysis												
System design												
Coding												
Testing												
Documentation												

Fig 3: Gantt Chart

6. Expected Outcomes

Upon completion, the system will deliver:

- ✧ Automated resume analysis and processing
- ✧ Intelligent candidate ranking based on job relevance
- ✧ Reduced manual workload in recruitment
- ✧ Improved accuracy and fairness in candidate selection
- ✧ Secure and user-friendly web application
- ✧ Scalable architecture for future enhancements
- ✧ Successful integration of machine learning into a real-world web system.

References

Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to information retrieval. Cambridge University Press. Retrieved April 3, 2026 <https://nlp.stanford.edu/IR-book/>

Ramos, J. (2003). Using TF-IDF to determine word relevance in document queries. Proceedings of the First Instructional Conference on Machine Learning. Retrieved April 3, 2026 <https://www.researchgate.net/publication/>